Data Science

Lecture 05: Neural Networks

Prof. Dr. Michael Granitzer

Speaker notes

Converted from LNKB-tex part-en-ds-chpM2-neural-networks.tex. Builds directly on Lecture 04: the perceptron is the linear classifier again, the delta rule is LMS again, and the sigmoid is logistic regression's again — each returning with a new purpose. Structured as a sequence of failures and their fixes.

Where We Left Off

Lecture 04 fixed the form — a line, a log-odds line — and chose the features by hand. Today relaxes both: stack simple units until the form is flexible, and let the network build its own features.

Lecture 04
a fixed form, hand-made features

→

Today
a learned form, learned features

→

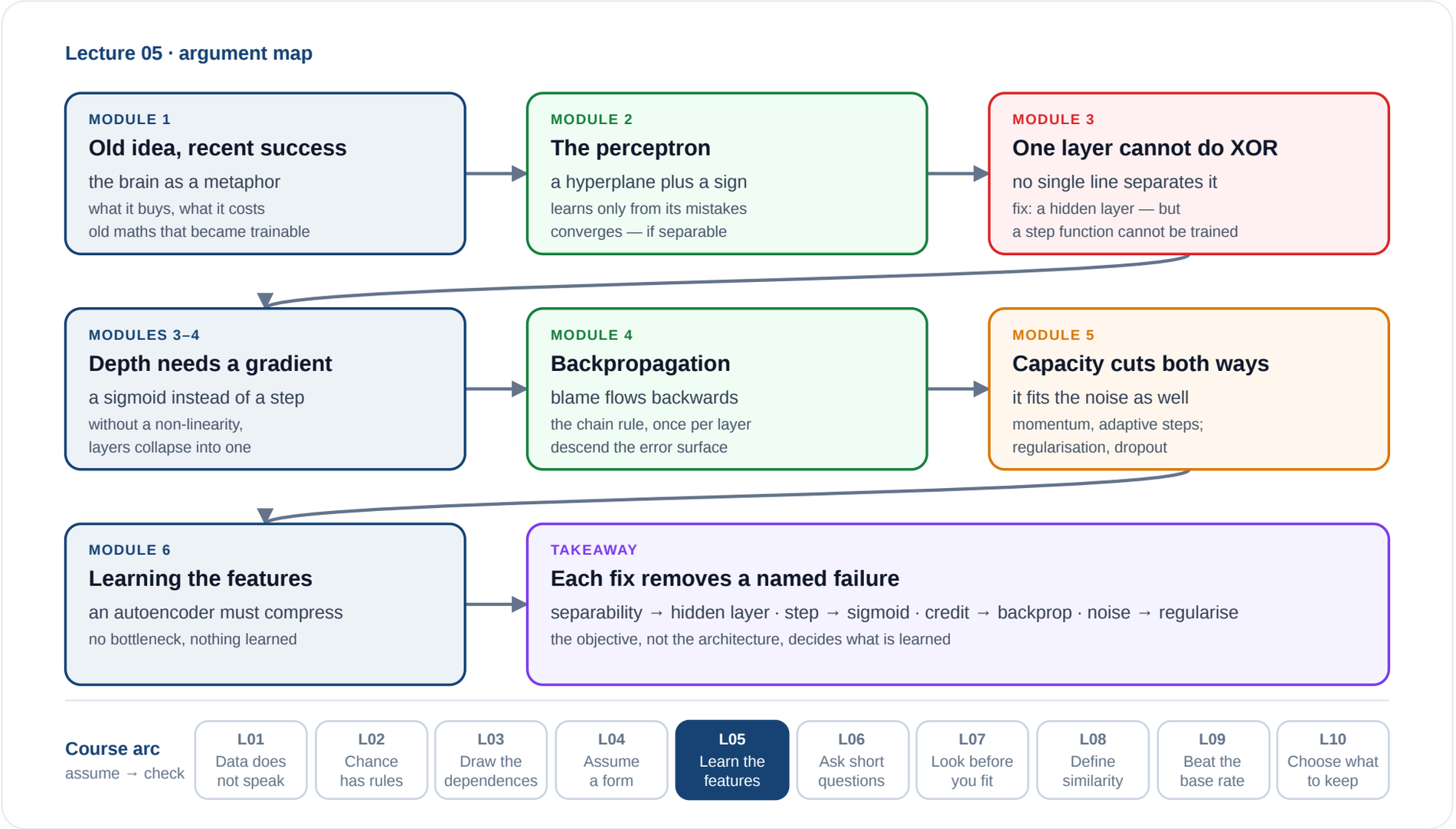
The price
capacity that also fits noise

> The perceptron is Lecture 04's linear classifier, the delta rule is LMS, and the sigmoid is logistic regression's — each returns with a new job.

Speaker notes

The lecture is a chain of failures and fixes. Keep the map on screen long enough to say "every box answers the box before it".

Argument Map



Speaker notes

Read it as failures and fixes: one hyperplane cannot do XOR, so add a hidden layer; a step function cannot be trained, so use a sigmoid; which weight is to blame is answered by backpropagation; capacity that fits noise is controlled by regularisation and dropout. The last module turns the network on its own input: an autoencoder learns features only because it is forced to compress. The strip: the assumption no longer sits in a fixed form, but in the architecture and the objective.

Module 1

Where the Metaphor Comes From

> ***Guiding Question:*** *The perceptron dates from 1958. What does the brain metaphor buy — and why did the idea start to work only around 2012?*

The Neuron

≈ 86 billion

neurons in the human brain (estimated, not counted)

10^{14}

synapses (estimated)

10^{3}–10^{4}

synapses on a typical neuron — about 10^{14} / 10^{11}

10^{-3} **s**

the fastest a neuron switches

A neuron receives signals through its dendrites, integrates them, and fires along its axon when the combined input passes a threshold.

Speaker notes

The neuron count is an estimate (Azevedo et al. 2009, ≈ 86 × 10^9), not a census. Per-neuron connectivity follows from the two big numbers: 10^{14} synapses over ~10^{11} neurons is ~10^3; cortical neurons typically carry 10^3–10^4, and only special cells (cerebellar Purkinje cells, ~10^5) reach far higher.

The switching time is the number to dwell on: a neuron is a million times slower than a transistor, yet the brain recognises a face in ~0.1 s — about a hundred sequential steps. Whatever it does, it does in parallel and shallowly.

What the Metaphor Buys — and What It Costs

Worth borrowing

Many simple units, massively connected; behaviour that emerges from weights rather than from rules; learning by adjusting connection strengths.

Not worth claiming

Artificial neurons are **not** models of biological ones. Backpropagation has no established biological counterpart, and no one is simulating a brain.

"Neural network" is a historical name, not a claim about neuroscience. Treat brain analogies in marketing material accordingly.

Speaker notes

Critique practice (P2, AI-context section): students meet brain analogies daily in press coverage of AI. Give them the distinction now so they can hear the difference between an architecture inspired by something and a model of it.

Takeaway

> The brain is the metaphor not for its speed — each neuron is a million times slower than a transistor — but for its **architecture**: many simple units, massively parallel, whose behaviour is learned by adjusting connection strengths. That is what artificial networks borrow, and all they borrow.

Speaker notes

This answers the module's opening question directly. The next unit asks the follow-up: if the architecture is that old, why did it only start to work recently?

The Idea Is Old; The Success Is Recent

The perceptron dates from 1958, backpropagation from the 1980s. The methods in this lecture are decades old. **What changed in 2012?**

More data

Large labelled datasets that did not previously exist (ImageNet, 2009).

More compute

GPUs able to run many passes over those datasets.

Better practice

Fixes available by 2012: ReLU activations, Glorot initialisation, dropout.

Followed the breakthrough rather than enabling it: ADAM (2014), batch normalisation and residual networks (both 2015).

Speaker notes

The guiding question of the module. Emphasise that no single item is a breakthrough in theory — the third column is a list of engineering fixes to problems that had made deep networks untrainable, which the rest of the lecture will meet one by one. Keep the chronology honest: AlexNet (2012) used ReLU, dropout, GPUs and ImageNet; ADAM, batch norm and ResNet came afterwards and are consequences of the success, not causes of it.

What This Implies

> The bottleneck was never the idea. It was **data, compute and a set of practical fixes** — which is why "we tried neural networks in 1995 and they did not work" was both true and misleading.

Mini-Exercise

A colleague says: "Deep learning was invented in 2012." What is wrong with the claim, and what did 2012 bring instead?

Mini-Exercise

A colleague says: "Deep learning was invented in 2012." What is wrong with the claim, and what did 2012 bring instead?

> The methods are decades old — perceptron 1958, backpropagation in the 1980s. 2012 brought the ability to **train them at scale**: data, GPUs and a few practical fixes came together.

Takeaway

> Neural networks are not new mathematics; they are old mathematics that became trainable. Every "new" technique below exists because a specific failure mode had to be removed.

Speaker notes

Set the reading frame for the whole lecture: each subsequent unit answers "what went wrong, and what fixed it". This makes the material a story rather than a catalogue.

Module 2

The Perceptron

> ***Guiding Question:*** *Least squares was nudged by examples it already classified correctly. What would a learning rule for decisions look like?*

The Model

A perceptron computes a weighted sum of its inputs and applies a **threshold**:

$$h_{\mathbf{w}}(\mathbf{x}) = \text{sign}\left(w_0 + \sum_{j=1}^{p} w_j x_j\right) = \text{sign}(\mathbf{w}^\top \mathbf{x})$$

Inputs

$x_1, \ldots, x_p$, plus a constant $x_0 = 1$ carrying the bias w_0

Weights

$w_0, w_1, \ldots, w_p$ — **the hypothesis is exactly this vector**

Output

$+1$ or -1: which side of the boundary the input lies on

This is the same object as *linear regression for classification* from Lecture 04 — a hyperplane and a sign. What is new is how the weights are found.

Speaker notes

Say the connection out loud (P7 spiral): students already know the geometry, so the only new content is the learning rule. The bias trick ($x_0 = 1$) is worth one sentence — it turns an affine boundary into a linear one in p+1 dimensions.

Weight Adaptation

Perceptron learning rule

For each training example $(\mathbf{x}, y)$ with $y \in \{-1, +1\}$:

$$\Delta w_j = \eta\big(y - h_{\mathbf{w}}(\mathbf{x})\big)x_j$$

Weights change **only when the example is misclassified**; correctly classified examples leave the hypothesis untouched.

Contrast with least squares, which is nudged by *every* example including the ones it already gets right. This is exactly the defect that made squared loss a poor classification loss.

Speaker notes

The comparison is the payoff of having taught regression-for-classification first. Ask what happens to the update when the prediction is correct: $y - h = 0$, so nothing moves. That is the fix, in one line.

The Perceptron Convergence Theorem

If the training data is **linearly separable**, the perceptron rule finds a separating hyperplane in a **finite** number of updates — regardless of the starting weights and of the learning rate.

What it promises

Termination with zero training error, guaranteed, when a solution exists.

What it does not promise

Anything at all when the data is **not** separable — the procedure simply never settles. And nothing about *which* separating hyperplane you get, or how it generalises.

Speaker notes

Math-first-then-meaning (P5). Both boxes matter: the theorem is strong and narrow. The "which hyperplane" gap is what support vector machines later close by asking for the maximum-margin one.

Mini-Exercise

A perceptron with $w_0 = -1.5$ and $w_1 = w_2 = 1$ receives binary inputs. Which logical function does it compute — and which w_0 turns it into OR?

Speaker notes

Doing AND and OR by hand makes the next unit's punchline land: they will try the same trick for XOR and fail, which is the historically decisive failure.

Mini-Exercise

A perceptron with $w_0 = -1.5$ and $w_1 = w_2 = 1$ receives binary inputs. Which logical function does it compute — and which w_0 turns it into OR?

> Sums $-1.5, -0.5, -0.5, +0.5$ for $(0,0), (0,1), (1,0), (1,1)$: only $(1,1)$ fires, so it is **AND**. With $w_0 = -0.5$ any single 1 clears the threshold: **OR**. The bias *is* the threshold, with its sign flipped.

Takeaway

> A perceptron is a **hyperplane plus a sign**, learned by correcting only its mistakes. Convergence is guaranteed — but only when a separating hyperplane exists at all.

Three Open Questions

Which solution?

Every separating hyperplane satisfies the theorem. The one you get depends on the initial weights and the order of the examples.

How long?

"Finite" can be very large. The bound grows as the **margin** shrinks — the distance from the separating hyperplane to the nearest training point.

And if not separable?

The procedure oscillates forever. Real data usually is not separable.

The theorem guarantees *a* separator on separable data — not which one, not how fast, not what happens otherwise. Zero training error is not the objective: a hyperplane with a tiny margin grazes the training points and may sit in the worst place for new ones.

Speaker notes

Connect back to L04: this is the training-error trap in a new guise. The margin defined here (distance to the nearest point) is the seed of SVMs and of why "any separator" is not good enough. For the curious: Novikoff's bound is at most $(R/\gamma)^2$ updates, with γ the margin and R the radius of the data.

Module 3

The Limit of One Layer

> ***Guiding Question:*** *You built AND and OR with a single perceptron. Why can you not build XOR — and what does that tell us about model families?*

The Question

You built AND and OR with a single perceptron. **Now build XOR:** output 1 when exactly one input is 1.

x_1	x_2	XOR
0	0	0
0	1	**1**
1	0	**1**
1	1	0

Try to find weights w_0, w_1, w_2. Take a minute.

Speaker notes

Let them actually try (P1/P2). The failure is the lesson, and it lands only if they have attempted it. Collect one or two candidate weight vectors and test them against the table together.

Why It Cannot Work

Plot the four points. The two positive cases sit on **opposite corners** of the square, and so do the two negative ones.
A perceptron can only draw **one straight line**. No single line puts $(0, 1)$ and $(1, 0)$ on one side while keeping $(0, 0)$ and $(1, 1)$ on the other.
XOR is **not linearly separable** — and no amount of training will change that.

> The limit is in the **hypothesis space**, not in the learning rule.

Speaker notes

This is the sharpest example in the course of a hypothesis space that simply does not contain the target function (P3, and the inductive-bias unit from L04). No optimiser can find what the model family cannot express.

The Consequence, Historically

Minsky and Papert's 1969 analysis of exactly this limitation is widely credited with the collapse of neural-network funding for over a decade.

> The criticism was mathematically correct and practically overdrawn: the fix — stack a second layer — was already known. What was missing was a way to **train** it.

Speaker notes

A good moment for epistemic humility (P2): a correct result, correctly proved, still misdirected a field for fifteen years because of what it was taken to imply. Ask what the analogous over-reading might be today.

The Fix: A Hidden Layer

XOR **is** expressible with two layers — for instance as $(x_1 \vee x_2) \wedge \neg(x_1 \wedge x_2)$: two perceptrons computing OR and NAND, and a third computing AND of their outputs.

> The hidden layer builds a **new representation** in which the problem *is* linearly separable. That is what depth buys: not more lines, but better coordinates to draw them in.

> A single perceptron is limited by what one hyperplane can separate. Stacking layers removes that limit — which is why the next question is how to train a stack at all.

Speaker notes

The insight box is the thesis of the entire lecture, and the phrase to leave on the board: *depth learns the representation*. It connects straight back to α, the representation function from L04, which was previously chosen by hand.

Why the Threshold Has to Go

The step function

$\text{sign}(\mathbf{w}^\top \mathbf{x})$ has derivative **zero everywhere** it is defined, and no derivative at all at the boundary.

The sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

smooth, bounded, and with a derivative that is non-zero where it matters.

Without a usable gradient there is no way to assign blame to a weight buried in a hidden layer. **Differentiability is what makes depth trainable.**

Speaker notes

The students met the sigmoid in logistic regression, so this is consolidation with a new motive (P7 spiral): there it made the output a probability, here it makes the network differentiable. Same function, different reason.

A Useful Property

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

The derivative is expressible in terms of the output — so a neuron that has already computed its activation can compute its own gradient for free.

> Both tails are flat: for large $|z|$ the derivative is nearly zero and learning stalls. This is the **vanishing gradient** problem, and it is why ReLU replaced the sigmoid in deep networks.

Speaker notes

The property is a computational convenience with real consequences for backpropagation's cost. The warning previews why the 2012 list included "new activation functions" — it was this failure being removed.

Mini-Exercise

$\sigma(0) = 0.5$ and $\sigma(12) \approx 0.99999$. Using $\sigma' = \sigma(1 - \sigma)$: how large is the gradient at $z = 0$, and at $z = 12$?

Mini-Exercise

$\sigma(0) = 0.5$ and $\sigma(12) \approx 0.99999$. Using $\sigma' = \sigma(1 - \sigma)$: how large is the gradient at $z = 0$, and at $z = 12$?

> $\sigma'(0) = 0.5 \times 0.5 = 0.25$, the steepest the sigmoid ever gets. $\sigma'(12) \approx 0.00001$: the update is effectively zero. The neuron is **saturated** and stops learning.

Takeaway

> The extra layer makes XOR *expressible*; the sigmoid makes the stack *trainable* — differentiability is the precondition for training depth. The price is saturating tails where learning stalls.

Speaker notes

The takeaway is the module's whole argument in one line, and it is worth writing on the board: expressible ≠ trainable. (Exact: $\sigma(12) = 0.999994$, $\sigma'(12) \approx 6 \times 10^{-6}$.)

Module 4

Depth, and How to Train It

> ***Guiding Question:*** *A hidden neuron has no target of its own. How much of the final error is it responsible for?*

Structure

Layer

$L_i = \{n_{1,i}, \ldots, n_{k,i}\}$ — a set of neurons. L_0 is the input layer.

Fully connected

Every neuron in L_{i-1} feeds every neuron in L_i.

Parameters

$\mathbf{W}_i \in \mathbb{R}^{|L_i| \times |L_{i-1}|}$ — one row of weights per neuron — and a bias vector $\mathbf{b}_i$, one bias per neuron.

The forward pass is then a chain of matrix products with a non-linearity between them:

$$\mathbf{a}_i = \sigma\big(\mathbf{W}_i \mathbf{a}_{i-1} + \mathbf{b}_i\big), \qquad \mathbf{a}_0 = \mathbf{x}$$

Speaker notes

Matrix form is worth the notation: it is how the computation is actually implemented, and it makes the next point obvious — without σ the whole chain collapses.

Why the Non-Linearity Is Not Optional

Without σ, a stack of layers is $\mathbf{W}_3\mathbf{W}_2\mathbf{W}_1\mathbf{x}$ (plus biases) — a **single matrix**, hence a single linear model. Depth without non-linearity buys nothing at all.

Each layer transforms the representation; the final layer draws a hyperplane in the coordinates the earlier layers built. This is XOR's fix, generalised.

Speaker notes

Ask students to multiply two 2×2 matrices and observe they get a 2×2 matrix. The argument is that short. It also explains why choosing an activation function is a modelling decision, not a detail.

Mini-Exercise

A network has 4 inputs, one hidden layer of 5 neurons and 3 outputs, fully connected, with a bias per neuron. How many parameters does it have?



Mini-Exercise

A network has 4 inputs, one hidden layer of 5 neurons and 3 outputs, fully connected, with a bias per neuron. How many parameters does it have?

$$\underbrace{(4 \times 5) + 5}_{\text{hidden layer}} + \underbrace{(5 \times 3) + 3}_{\text{output layer}} = 25 + 18 = 43$$

> Counts grow with the **product** of adjacent layer sizes, which is why parameter counts explode with width.

Takeaway

A multilayer network is matrix products separated by non-linearities. **The non-linearity is what makes depth mean anything**, and each hidden layer is a learned change of coordinates.

Speaker notes

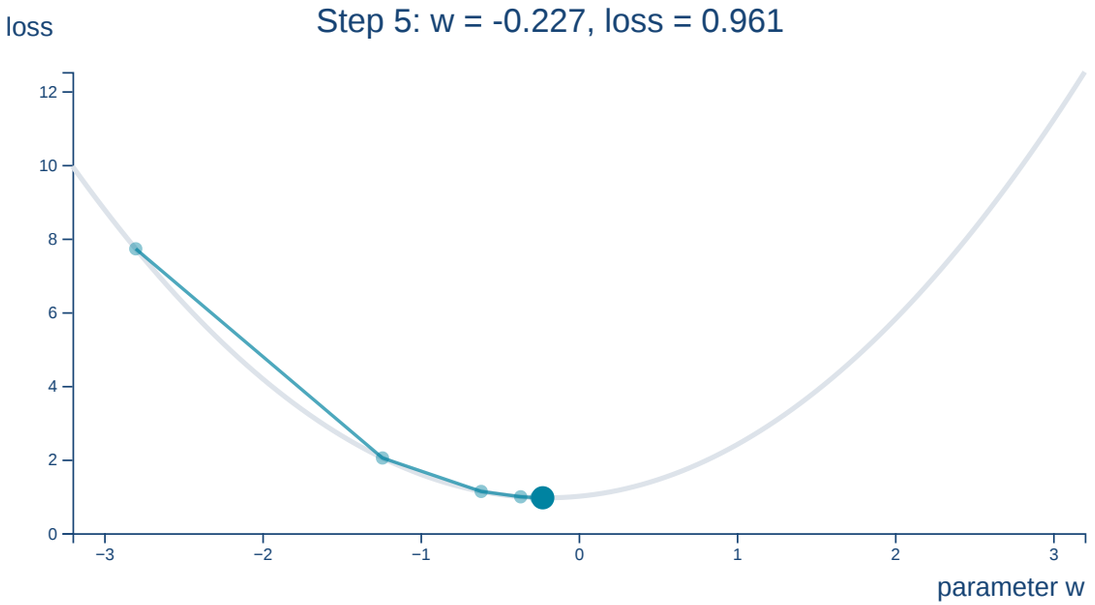
Connect the parameter count to L04's bias–variance discussion: capacity grows fast, so the regularisation unit later is not optional bookkeeping.

The Error Surface

For weights $\mathbf{w}$ and training set D, the squared error

$$E(\mathbf{w}) = \frac{1}{2} \sum_{(\mathbf{x},y)\in D} \big(y - h_{\mathbf{w}}(\mathbf{x})\big)^2$$

is a **surface over the weight space**. Training means walking downhill on it.



Speaker notes

The same visualization as in L04's optimisation unit, deliberately: students should recognise that nothing new is happening — only the function being minimised has changed. Ask what the axes are now (weights, not a single parameter) and what the picture would look like in 10^6 dimensions.

Two Ways to Descend

Batch gradient descent

Accumulate the gradient over **all** examples, then take one step.

Stable, expensive per step, and it needs the whole dataset in memory.

Incremental (stochastic) gradient descent

Update after **each** example.

Noisier, far cheaper per step, and the noise can help escape poor regions.

The **delta rule**, $\Delta w_j = \eta(y - \mathbf{w}^\top \mathbf{x})x_j$, is incremental gradient descent on the squared error — the LMS rule from Lecture 04 under a new name.

Speaker notes

Three names for one thing (LMS, delta rule, incremental gradient descent) is a genuine source of confusion in the literature; naming them together once saves students later grief. Modern practice sits between the two: mini-batches.

Mini-Exercise

You train on 1,000,000 examples, and one pass over them takes an hour. How many weight updates per hour does **batch** gradient descent make — and how many does **incremental**?

Speaker notes

The arithmetic makes the case for stochastic methods far better than an assertion does. The risk links back to the learning-rate comparison in L04.

Mini-Exercise

You train on 1,000,000 examples, and one pass over them takes an hour. How many weight updates per hour does **batch** gradient descent make — and how many does **incremental**?

> Batch: **one**. Incremental: **one million**, each on a noisy estimate of the gradient. Progress per unit of computation is vastly higher — at the risk of instability if η is too large.

Takeaway

Training is descent on an error surface. **Batch** steps are accurate and rare; **incremental** steps are noisy and frequent — and on real data, frequency wins.

The Credit Assignment Problem

The output layer's error is measurable: we know the target. But a hidden neuron has **no target of its own**.

> *How much of the final error is this hidden weight responsible for?*

Speaker notes

Guiding question of the module (P1). This is the question that stalled the field for a decade — the XOR fix was known, the training method was not.

The Answer: Blame Flows Backwards

Output layer

The error is measured against the target, times the slope of the activation: $\delta_o = (y - o)\sigma'(z_o)$.

Hidden layers

The error of a neuron is the **weighted sum of the errors of the neurons it feeds**, times its own slope: $\delta_h = \sigma'(z_h) \sum_k w_{hk}\delta_k$.

A neuron that contributed strongly to a wrong answer receives a large share of the blame. That is the whole idea; the calculus is the chain rule applied layer by layer.

Speaker notes

Give the sentence before the formula (P4). Students who leave with "blame is distributed backwards in proportion to contribution" have the concept, even if the derivation fades.

The Algorithm

Backpropagation (incremental form)

repeat **for each** $(\mathbf{x}, y) \in D$: **forward:** compute activations layer by layer up to the output **backward:** compute the output error, then propagate it to each earlier layer **update:** $w \leftarrow w + \eta \cdot \delta_{\text{target}} \cdot a_{\text{source}}$ **until** convergence **or** $t > t_{\max}$

The backward pass costs about **twice** the forward pass, so one training step costs about **three** times inference — the fact behind every training-cost estimate you read.

Speaker notes

The update rule has the same shape as LMS and the delta rule: error × input, with a plus sign because δ is defined from y − o (δ = −∂E/∂z). Point that out — it is the third appearance of one pattern, and recognising it is worth more than the derivation.

Mini-Exercise

A hidden neuron feeds two outputs through weights 0.8 and -0.2; their errors are $\delta_1 = 0.5$ and $\delta_2 = 1.0$. What is its propagated error, before the σ' factor?

Speaker notes

The cancellation surprises students who expect blame to accumulate monotonically. Follow-up if time allows: multiply by $\sigma'(z_h) \le 0.25$, and note that the weight update also scales with the source activation — a near-zero activation barely learns, the seed of the vanishing-gradient discussion.

Mini-Exercise

A hidden neuron feeds two outputs through weights 0.8 and -0.2; their errors are $\delta_1 = 0.5$ and $\delta_2 = 1.0$. What is its propagated error, before the σ' factor?

> $0.8 \times 0.5 + (-0.2) \times 1.0 = 0.4 - 0.2 = 0.2$. Blame is weighted by **weight × error**: output 1 contributes more, and the negative weight partly cancels it.

Takeaway

Backpropagation solves **credit assignment**: error flows backwards, weighted by the connections that carried the signal forward and scaled by each neuron's slope σ'. It is the chain rule, applied once per layer — no more and no less.

Module 5

Making Training Work

> ***Guiding Question:*** *The network fits the training data perfectly and fails on new data. Which of your assumptions was wrong?*

Two Failure Modes

Local minima

A point lowest in its neighbourhood, $E(\mathbf{w}_{\text{local}}) \leq E(\mathbf{w})$ for all $\mathbf{w}$ near it — yet not globally: $E(\mathbf{w}_{\text{opt}}) < E(\mathbf{w}_{\text{local}})$.

In practice, large networks appear to have **many** local minima, most of them close in value to the best one — so this matters less than intuition suggests.

Flat regions

Near-zero gradients produce near-zero steps. Training appears to have converged when it has merely stalled — the sigmoid's saturated tails are the usual cause.

Speaker notes

The local-minima nuance is worth stating carefully: the textbook fear turned out to be largely misplaced for high-dimensional networks, while flat regions and saddle points are the real obstacle. This is a case where the field's own intuition changed.

Momentum

$$\Delta \mathbf{w}_t = -\eta \nabla E(\mathbf{w}_t) + \alpha \Delta \mathbf{w}_{t-1}$$

Each step keeps a fraction α of the previous one: consistent directions accumulate speed, oscillating directions cancel.

> Momentum helps through flat regions and damps zig-zagging in narrow valleys — but the learning rate η remains critical for convergence.

Speaker notes

The rolling-ball metaphor is standard and accurate enough. Ask which of the two failure modes momentum addresses (flat regions, partially) and which it does not (a genuinely bad minimum).

Adaptive Learning Rates

AdaGrad

One learning rate **per parameter**: η divided by the square root of that parameter's accumulated squared gradients.

Parameters with large gradients slow down quickly; rarely-updated parameters keep a large step size.

And its successors

RMSProp replaces the ever-growing sum with a moving average; **ADAM** combines that with momentum, and is the common default today.

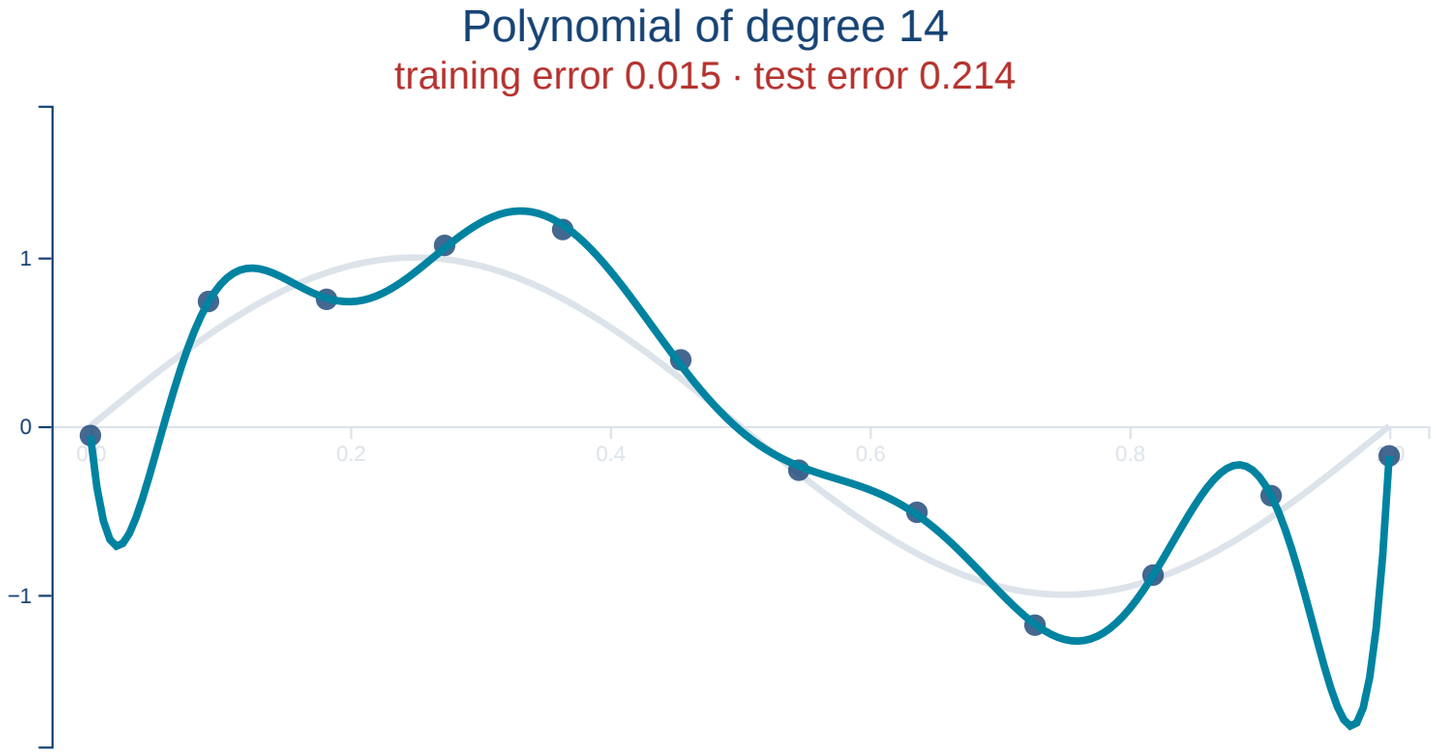
The obstacles are flat regions and badly scaled parameters, not mythical local minima. **Momentum** adds inertia, **adaptive methods** give each parameter its own step size — and both exist to remove a specific, diagnosable failure.

Speaker notes

Return to the framing from the history unit: every optimiser on this slide is a fix for an observed pathology. Students should be able to name the pathology each one addresses.

Capacity Cuts Both Ways

A network with enough neurons can fit **any** training set — including its noise. The decision surface becomes intricate where the data is sparse.



Speaker notes

Stepped viz (P6): each click raises model capacity. Ask for a prediction before each step — training error falls monotonically (0.55 → 0.02), but test error bottoms out at degree 5 and then climbs. The gap between the two curves *is* overfitting, made visible. Polynomials stand in for network capacity; the phenomenon is identical.

Regularization

Add a penalty on the weights to the loss:

$$E_{\text{reg}}(\mathbf{w}) = E(\mathbf{w}) + \lambda\Omega(\mathbf{w})$$

where λ controls how much the penalty counts.

L_2 — $\Omega = \|\mathbf{w}\|_2^2 = \sum_j w_j^2$

Penalises **peaky** weight vectors and prefers diffuse ones: every input gets used a little. The default choice.

L_1 — $\Omega = \|\mathbf{w}\|_1 = \sum_j |w_j|$

Produces **sparse** weights (many exact zeros): a neuron keeps only its most important inputs — a feature selector.

Speaker notes

The conceptual payoff (stated in the takeaway) links three lectures: a regulariser writes background knowledge into the objective — "prefer simple explanations", "expect most inputs to be irrelevant". It is the inductive bias from Lecture 04, made explicit and tunable. Bias is not an accident to be minimised, it is a choice to be stated. λ is the dial between fitting the data and honouring the assumption.

Dropout

During training, each neuron's output is **silenced with probability** $1 - p$. The network cannot rely on any single unit, so it must spread the representation across many.
Because training silences a fraction of the units, a layer's total output is smaller during training than at inference. The surviving activations are therefore scaled by $\frac{1}{p}$ to compensate.

Dropout is active during **training only**. Forgetting to disable it at inference time is a classic bug.

Speaker notes

The scaling detail is exactly the kind of thing that is skipped and then debugged for a day. Ask why the network "cannot rely on a single unit" — it is co-adaptation being broken deliberately.

Mini-Exercise

Your network reaches 99% training accuracy and 71% validation accuracy. What is the diagnosis, and which intervention would you try first?



Speaker notes

Reward reasoning about cost and assumptions rather than reciting a list. Note that "smaller network" is the honest option students under-use.

Mini-Exercise

Your network reaches 99% training accuracy and 71% validation accuracy. What is the diagnosis, and which intervention would you try first?

> **Overfitting** — a 28-point gap. More data if obtainable: it reduces variance without adding assumptions. Otherwise stronger regularisation (λ, dropout), a smaller network or early stopping — cheap to try, and the gap shows whether it worked.

Takeaway

Capacity that can fit the signal can also fit the noise. **Regularisation is inductive bias made explicit**: it encodes what you believe about the unseen data — and the gap between training and validation error is how you find out whether you believed correctly.

Module 6

Learning the Representation

> ***Guiding Question:*** *In Lecture 04 we chose the features by hand. Can a network choose them for us — and how would we know if it did?*

Learning to Reproduce the Input

An **autoencoder** is trained so that its output reproduces its input, through a hidden layer that is **narrower** than the input.

Encoder

maps the input to the narrow hidden layer

Bottleneck

a compressed code — the representation being learned

Decoder

reconstructs the input from that code

No labels are required: the target *is* the input.

Speaker notes

This is the first unsupervised method in the course. The label-free framing is worth emphasising — it is the mechanism behind pretraining, and behind "self-supervised" methods students will hear about constantly.

What Makes It Work

The bottleneck does the work

A narrow hidden layer is a **lossy compression** of the input: the network must discard whatever it can afford to lose.

Too wide and it cheats

If the hidden layer is large enough, the network simply learns the **identity function** and has represented nothing.

Speaker notes

The "too wide and it cheats" failure is the interesting one: an objective that is trivially satisfiable teaches nothing. Ask what the analogous failure is for a language model trained to copy its input.

Two Refinements

Denoising autoencoders

Corrupt the input, then require reconstruction of the *clean* version — the identity function is no longer a solution.

Weight sharing

Tie the decoder to the encoder, $\mathbf{W}_{1,2} = \mathbf{W}_{0,1}^{\top}$, halving the weights to learn.

For binary inputs, cross-entropy replaces squared error as the reconstruction loss — the same substitution as in logistic regression.

How Would We Know It Worked?

A low training reconstruction error proves little — the identity function has none. Two checks do:

Held-out reconstruction

Reconstruction error on data the network never saw: the code must generalise, not memorise.

Downstream use

Train a simple model on the code, with few labels. If it beats the same model on the raw inputs, the representation has captured structure.

A learned representation is judged like any model: on unseen data, and by what it lets us do — not by how well it fits its own training set.

Speaker notes

This answers the second half of the module's guiding question. The downstream test is how pretrained representations are evaluated in practice ("linear probing"). A quick visual check is also useful: nearest neighbours in code space should look similar to a human.

Mini-Exercise

An autoencoder with 784 inputs and a 784-neuron hidden layer reaches almost zero reconstruction error. Is this a good result?

Mini-Exercise

An autoencoder with 784 inputs and a 784-neuron hidden layer reaches almost zero reconstruction error. Is this a good result?

> No — a warning sign. With no bottleneck it has probably learned the **identity**: copy the input across, represent nothing. Narrow the bottleneck, or corrupt the input (denoising), so copying no longer solves the task.

Takeaway

An autoencoder learns a representation by being **forced to lose information**, and is judged on unseen data and downstream use. Remove the constraint and it learns nothing — the objective, not the architecture, decides what a model learns.

Speaker notes

Close the lecture on the through-line: L04 chose the representation by hand (the function α); here the network learns it, and the quality of what it learns is determined entirely by the pressure the objective puts on it.